Contextual Prediction and Self-Certification

Jeremy H. Carroll

Conditional limits and their role in safe recursive self-modification

2026-10-23

Version 1.8.3

Abstract

A reasoner can represent contextual statistics without possessing a single joint distribution that reproduces them. It can check a ledger without establishing that no entries have been withheld. It can satisfy a provenance policy without establishing that a proposed action is safe. These are different limitations, with different hypotheses. This paper accompanies *Relational Reasoning, Memory, and Embedded Knowledge*, with each limitation attached to its own observation, task and resource assumptions. We carry forward the finite support, observation-fiber, ledger, relabeling, and three-role results of the epistemic-contextuality precursor, with their executable checks. We add a Lean proof that preservation of observation fibers by every permitted update is equivalent to preservation under every finite update word. This supplies a precise sufficient condition for the precursor's unresolved temporal-privacy claim. We also exhibit why a pinned ledger tip needs an integrity assumption and why a passing provenance packet supplies no action-safety guarantee. The finite contextuality LP now has a Lean proof of primal and dual attainment, including the canonical maximum over deterministic columns and the joint-or-positive-defect alternative. A standalone exact-rational two-phase simplex solver returns independently replayed certificates. Five solver outputs are checked numerically inside Lean, and rational certificates are proved sound against all real probability vectors. The package also includes an executable Lean two-phase simplex with exact pivots, a shared finite pivot budget, and an independently checked return path. Accepted optimal results are feasible and globally optimal. The solver also extracts independently checked Farkas infeasibility witnesses and feasible descent rays; their soundness over real vectors is formalized in Lean. Its recursion is total with budget exhaustion represented as an inconclusive refusal. Conditional trajectory-error and correction-window results connect a declared residual enclosure to an observation uncertainty margin. An integrated partial-observation account separates whole-agent retention from reports, formalizes maximally permissive sound report checks and score-based decision guarantees, and adds exact scalar minimax certificates, memory-state capacity, total-variation loss transfer, and a terminating binary-query recurrence equivalent to the existence of a valid bounded-depth plan. A source-checked persistent memory now enters the repository agent context before dispatch. Separate-process live trials demonstrate exact reuse of a learned observation and retain an arithmetic failure after correct recall. These finite results distinguish reliable memory access from reliable reasoning.

1. Two papers, two questions

The current RSI companion, version 1.2.3, develops record-relative knowledge, future-preserving memory, finite-carrier limits, and executable checked append, recall and invariant-preserving gates. This version 1.8.3 develops contextual representation, internal checks, quantitative recovery, statistical experiment simulation and observation planning. Both integrate the October 22 memory and embedded-knowledge study. A source-lineage entry for a separate historical recovery edition 1.4.0 remains historical; it does not supply the current proof estate. Both now include a persistent memory implementation, a repository agent-context adapter, and captured live restart trials with explicit limits.

The distinction matters mathematically. A fiber collision concerns two states with the same observation and different task answers. A contextuality obstruction concerns the nonexistence of a probability distribution whose marginals equal several context-indexed distributions. Neither is the first cohomology of a cover alone. Likewise, a proof of recovery error does not issue a permission, and a permission does not supply a proof of safety.

This edition succeeds *Conditional Limits on Global Prediction, Self-Knowledge and Self-Certification for Physical Reasoners*, version 1.2.0, formerly filed under `001_finite_obstruction_calculus/002_asi_no_go_theo-`

`rem_epistemic_contextuality`. The redundant original directory has been retired. Its monograph, README, mechanization map, references, correction log and superseded floating-point listings are retained verbatim as archival documents under `docs/precursor_*`; they are not the current manuscript. The source disposition is recorded in `docs/source_consolidation.json`. The source inventory and the exact RSI companion release digest are in `docs/source_lineage.json`. The inherited Lean namespace is retained to make source comparison straightforward.

1.1 Status of the claims

Formal means a named declaration checked by the deposited Lean package. **Exact check** means a specified finite computation using integers or rational arithmetic. **Mathematical argument** means a proof in the text without a complete Lean formalization. **Imported** means a cited theorem not proved here. **Policy** means behavior follows from the adopted gate definition. **Open application** means the needed premise about a real system has not been established.

No result here bounds arbitrary intelligence, arbitrary computation, or the possibility of recursive improvement. No result turns an observation boundary into permanent privacy without a dynamics premise. No result derives the chosen authority policy from physical law.

2. Contextual prediction

Let X be a finite set of measurements, each with a finite nonempty outcome set. A context is a jointly permitted subset of measurements. For each context C, an empirical model supplies a normalized nonnegative rational distribution e_C on its outcome assignments. Assume compatible marginals on context overlaps. This assumption is often called no-signalling; it is a property of the supplied tables, not a consequence of finiteness.

Let G be the finite nonempty set of global outcome assignments. A probability vector p on G induces all context marginals by a linear map A. A noncontextual representation is a probability vector satisfying

$$Ap = e, \qquad p_g \geq 0, \qquad \sum_{g \in G} p_g = 1.$$

Here e is the concatenation of the context tables. An assignment and a distribution over assignments are different objects: absence of a supported assignment implies absence of a representing probability distribution, but a supported assignment alone does not establish agreement with all probabilities.

2.1 The finite rational dichotomy

Theorem 1 (formal over the reals, including rational data by scalar embedding). Exactly one of the following holds: a representing joint probability vector exists, or the minimum mismatch is strictly positive:

$$d(e) = \min_{p \in \Delta(G)} \|Ap - e\|_1 > 0.$$

The deposited proof constructs both an attaining probability vector and an attaining dual pair. It does not assume a zero-gap certificate to establish existence. The probability simplex is nonempty and compact; its image under the marginal map is compact and convex. The proof first obtains a nearest point. If the empirical model belongs to that image, the zero functional supplies the dual optimum. Otherwise, geometric Hahn–Banach separation of the image from the open ball at the minimum distance yields a supporting functional. Its operator norm supplies the normalization. In the finite ℓ^1 space, its values on coordinate basis vectors give the bounded dual coefficients.

The canonical dual certificate is

$$\|y\|_\infty \le 1, \qquad \langle y, e\rangle - \max_{g\in G}\langle y, A\delta_g\rangle = d(e).$$

The formal theorem `canonical_dual_attainment` exhibits a deterministic column attaining that finite maximum. The offset version returns a scalar satisfying

$$\langle y, A\delta_g\rangle + z \le 0 \quad (g \in G), \qquad \langle y, e\rangle + z = d(e).$$

Weak duality holds over any ordered field, including the rationals: a feasible dual value is no larger than the mismatch of any probability vector. Equality therefore proves optimality. `dual_solution_optimal` also proves that no feasible dual pair has a larger objective. `joint_or_positive_dual` and `positive_dual_excludes_joint` give the exhaustive and mutually exclusive contextuality alternative.

2.2 The exact-rational standard-form solver

The standalone solver uses a two-phase simplex with Bland selection [7] and variables for probability weights and positive and negative residuals:

$$Ap - u + v = e, \qquad p, u, v \ge 0, \qquad \sum_g p_g = 1, \qquad \min \sum_i (u_i + v_i).$$

`slacks_l1_lower_bound` proves that every feasible residual split upper-bounds the ℓ^1 cost. `exact_slacks_exist` constructs a split attaining that cost. Thus the standard-form problem implemented by the solver has exactly the same optimum, and `contextuality_standard_form_strong_duality` supplies attaining primal and dual solutions for that literal formulation.

The Python implementation adapts the repository's two-phase Bland simplex, using only integers and `Fraction`. Phase one introduces artificial variables and detects infeasibility; phase two optimizes the original objective. One declared pivot budget covers both phases and removal of artificial basic variables. The contextuality reduction is always feasible and bounded for nonempty finite columns; general standard-form inputs can also receive checked infeasible or unbounded outcomes. Exhaustion of a resource budget is a refusal, never a contextuality verdict. No floating approximation or rational reconstruction is used.

A separate checker rebuilds the standard-form constraints and checks primal nonnegativity, every equality, every dual inequality, equality of objectives, the residual vector, the witness norm, the offset, and their binding to the displayed result. The pivot count is diagnostic and is not a premise of the mathematical certificate. Input validation checks normalization and overlap marginals before the contextuality LP; enumeration and dense-matrix caps are checked before allocation.

Rational zero-gap certificates are proved sound over the rationals and, by scalar embedding, against all real probability vectors. Five concrete solver outputs are translated into `RationalCertificates.lean`; Lean checks all their numerical constraints and the claimed defects, then applies that general result. These are the classical mixture (0), PR box (2), near-Tsirelson table (4/5), an odd-parity triangle (2), and a Bell table just above the classical boundary (1/25000000). `docs/lp_certificates.json` contains the exact data, with source bindings; its generator has a refusing check mode.

2.2.1 Executable simplex correctness and finite-budget termination

`Simplex.lean` implements the two phases, negative-right-hand-side normalization, artificial-variable initialization, least-index entering selection, ratio selection with least-basic-index tie breaking, exact row elimination, artificial-basis cleanup, removal of redundant active rows, phase-two objective reconstruction, primal and dual extraction, and independent final replay. `CertifiedOutcomes.lean` extends those same pivot, initialization and cleanup primitives with terminal-tableau retention and checked infeasible/unbounded returns.

`SimplexDriver.lean` runs the extended `solveCertified` entry point on exact JSON inputs. The original `Simplex.solve` remains the optimal-only compatibility model. Table rows are materialized as finite vectors after each pivot, avoiding exponential reevaluation of earlier functional updates.

Theorem 1a (formal, successful-return correctness). For every finite rational standard-form problem and every natural pivot budget, a successful Lean `solve` return satisfies primal nonnegativity, every equality, dual feasibility, and equality of both objectives to the returned value. Every feasible rational competitor has objective at least this value. `valid_is_real_optimal` also proves that a rational certificate lower-bounds every feasible real competitor for the same rational data. The final checker is part of the algorithm; the proof does not assume that the unverified extracted tableau is correct.

The pivot development separately proves normalization of the entering column, equivalence of the represented linear equations before and after a nonzero pivot, preservation of the objective on those equations, and nonnegative right-hand sides under the ratio-test hypotheses. Initialization gives nonnegative right-hand sides. These are algebraic invariants for the actual pivot definition, including degenerate zero right-hand sides.

Theorem 1b (formal, shared-budget accounting). Both simplex recursions are structural on the remaining pivot budget; cleanup is structural on the finite original row list. Every pivot consumes exactly one unit. `simplex_budget_exact`, `cleanup_budget_exact`, and `execute_budget_exact` prove that the successful execution trace length plus the remaining budget equals the original budget. `accepted_pivots_match_trace` identifies the displayed count with the actual trace; `solve_pivot_bound` bounds it by the supplied budget. Thus the optimal-only Lean entry point terminates with an accepted optimum or refusal for every finite typed input and budget. The extended search is likewise structural on the budget; `search_budget_exact` covers both optimal and ray terminal tables. Cleanup still consumes the same remaining budget. The extended entry point returns a checked optimum, checked infeasibility, checked unboundedness, or an inconclusive refusal. This is a finite-pivot termination guarantee, not a wall-clock bound on rational bit complexity or operating-system resources.

The Python algorithm and the executable Lean algorithm are compared on complete outputs: every primal coordinate, every dual coordinate, exact value, pivot count, refusal kind, Farkas coordinate and ray coordinate. Cases include all five contextuality fixtures at budgets just below and at completion, redundant rows, negative right-hand sides, empty problems, degeneracy, and the classical cycling example under Bland selection. These finite comparisons do not prove universal Python-to-Lean refinement. Budget exhaustion and certificate mismatch remain inconclusive; a mathematical outcome requires the independent checker described next. A proof that unbudgeted Bland execution always finds an optimum, a complete phase-invariant proof removing the final checker, and universal cross-language refinement remain outside this edition.

The supplied review's input-boundary repair is implemented: ordinary list/tuple containers and integer context indices are checked before sorting or hashing. The inherited `is_distribution` and `marginalises` helpers are partial checks of supplied rows and marginal equalities respectively; they are not complete probability-model validators. Current public contextuality evaluation checks positivity, normalization, coverage and overlap compatibility.

2.2.2 Checked infeasibility and unboundedness

Consider the literal standard-form problem

$$\min c^T x, \qquad Ax = b, \qquad x \geq 0.$$

Theorem 1c (formal, infeasibility certificate). A rational vector satisfying

$$A^T y \geq 0, \qquad b^T y < 0$$

excludes every real feasible vector. For a feasible nonnegative vector the equality of the two ways of pairing the constraints gives a nonnegative value, contradicting the strictly negative pairing with the right-hand

side. No sign restriction is imposed on the witness itself because the original constraints are equalities. `infeasible_excludes_real` proves this using only the certificate inequalities and the original rational matrix, embedded in the reals.

Theorem 1d (formal, unboundedness certificate). A rational feasible base point and descent direction satisfying

$$Ax_0 = b, \quad x_0 \geq 0, \quad d \geq 0, \quad Ad = 0, \quad c^T d < 0$$

produce the feasible ray and objective

$$x(t) = x_0 + td, \qquad c^T x(t) = c^T x_0 + tc^T d \quad (t \geq 0).$$

`ray_real_feasible` and `ray_real_objective` prove these statements for every nonnegative real parameter. `unbounded_below_real` constructs a real feasible vector below every real objective floor. Checking a direction alone would be insufficient: the base point is essential, including when the equality constraints are inconsistent.

The Python interface preserves the existing optimal return type. `CertifiedInfeasible` and `CertifiedUnbounded` subclass `Refusal` for compatibility with callers that catch all unsuccessful optimization attempts. Their `.certificate` fields carry the frozen typed witnesses. `verify_infeasible` and `verify_unbounded` replay only original input constraints, without pivot traces or tableau state. They validate dimensions, exact number types and strict signs; booleans, floats and malformed witnesses are rejected. Pivot metadata is diagnostic and is not a mathematical premise. The successful contextuality reduction still returns its usual zero-gap certificate.

Phase one obtains its witness from the transformed artificial columns with the original row signs restored. Phase two retains the entering column when no leaving row exists, extracts the current feasible base and nullspace direction, and replays both against the original matrix. Redundant-row deletion never changes what the checker validates. A failed replay yields `CERTIFICATE_MISMATCH`; phase-one apparent unboundedness also refuses rather than being promoted. In Lean, `accept_correct` and `solve_certified_correct` guard all three mathematical outcome constructors. `solve_infeasible_correct`, `solve_unbounded_correct` and `solve_optimal_correct` apply the general real-vector soundness results to actual accepted returns.

The extended execution suite compares 95 complete Lean/Python outputs, including signed and scaled Farkas vectors, nonzero ray bases, redundant rows, empty dimensions and zero-pivot outcomes. Certificate mutation tests independently change inputs, witness signs, base points, nullspace directions, strict objective inequalities and metadata; forced checker rejection confirms that extraction alone cannot issue a mathematical outcome. This is finite execution evidence, not universal Python refinement or proof that every finite budget suffices. The certificate forms are standard [8]; the contribution here is their checked implementation and scoped mechanization.

2.3 The PR support obstruction

For binary settings and outcomes, the PR support requires

$$a_x \text{ xor } b_y = x \text{ and } y.$$

The three contexts other than the final one require equal outcomes. Together they imply equality in the final context, whose rule instead requires unequal outcomes. Thus no global assignment satisfies every context. Any positive-weight entry of a nonnegative joint representing the PR tables would have to satisfy every support constraint: otherwise it would contribute positive mass to a zero-probability outcome. This is impossible.

`ResultI` proves the support contradiction and its consequence for finitely supported natural-weight measures. Its natural weights are a support model, not a Lean construction of arbitrary real probability measures. The

positive-weight argument above extends to finite probability vectors in prose. The same module verifies exact marginal weights for the equal mixture of the all-zero and all-one assignments, supplying a classical global section.

The Python checks use `Fraction` to verify the PR table, the classical mixture, and a full-support table with weights 17/40 and 3/40, exchanged in the final context. They enumerate all sixteen deterministic assignments and obtain the classical CHSH upper bound 2. The full-support table has CHSH value 14/5, so it cannot be a convex combination of those deterministic assignments. It nevertheless has supported global assignments. This separates probability incompatibility from absence of global support. A further rational probe lies just above the classical boundary; no floating tolerance is used.

A context-aware predictor may store the whole empirical model and answer from the selected context. Theorem 1 excludes a particular joint representation. It does not exclude storage of contextual statistics or a world-model containing them. The first cohomology of the context-cover nerve is a property of the cover; it cannot distinguish two empirical models on the same cover. A support-presheaf obstruction is a different construction and is not implemented here.

3. Observation fibers and future privacy

Let $\pi : S \to B$ be the information supplied to a predictor and let $q : S \to Q$ be its task. If two states have the same report and different task answers, no report-reading function recovers the task on both states:

$$\pi(s) = \pi(t), \quad q(s) \neq q(t) \quad \Longrightarrow \quad \neg\exists f\ \forall z,\ f(\pi(z)) = q(z).$$

`ResultII.fiber_lemma` proves this for arbitrary types. Its finite witness observes the first coordinate of a pair of bits. Full-state prediction fails because the hidden coordinate can vary. With swap dynamics, however, the hidden coordinate becomes the next report. The source's inference from instantaneous information loss to indefinite privacy is therefore false. The witness is preserved in both Lean and Python.

3.1 The missing dynamics premise

Let U be a type of permitted updates and let $F_u : S \to S$ be the update indexed by u. Apply a word from left to right, with the empty word acting as identity. Define fiber preservation by

$$\forall u, s, t, \quad \pi(s) = \pi(t) \Longrightarrow \pi(F_u(s)) = \pi(F_u(t)).$$

Theorem 2 (formal). Every permitted update preserves observation fibers if and only if, for every pair in the same initial fiber and every finite update word, the final reports agree.

Proof. For the forward direction, induct on the word. The empty case is the initial equality. For a nonempty word, the first update preserves equality, and the induction hypothesis applies to the remaining word. For the reverse direction use a word of length one. `Bridge.fibers_preserved_iff_all_words` checks both directions in Lean without Mathlib.

An intertwining family supplies this premise:

$$\pi \circ F_u = \overline{F}_u \circ \pi.$$

Theorem 2 therefore gives a precise sufficient condition for privacy of distinctions within a report fiber under the declared updates. It also covers adaptive selection of updates by a deterministic policy reading only the common transcript: equal transcripts select the same next update, preserving the induction. The adaptive extension is a prose argument, not a separate Lean theorem. Controllers with additional private observations, unmodeled interventions, or changed report maps need additional premises.

The RSI companion treats the more general task of preserving every future task readout, defines its greatest update-stable observational equivalence, and studies finite memory refinements and recovery. The theorem

here is the report-fiber specialization that connects the precursor's counterexample to that construction. It does not import the companion's proof modules into this package, and the two packages build independently.

3.2 Projection Obstruction: error propagation and correction windows

The supplied review and a fresh reading of Projection Obstruction supply useful conditional boundaries. They do not identify contextuality defect with a quantum product-retraction residual. The former is distance to a convex probability polytope; the latter is the residual of a specified nonlinear marginal-product retraction. PO-034 section 4.1 shows explicitly that this retraction need not be a nearest-product projection. PO-000 also exhibits vanishing pairwise defects with a positive global obstruction, so a locally zero diagnostic is not a global error certificate.

For an actual exact update and an implemented trajectory, suppose the update is Lipschitz with factor L, the initial error is bounded by E0, and every actual one-step implementation defect is bounded by r_k. Then:

$$E_{k+1} \le LE_k + r_k, \qquad E_n \le L^n E_0 + \sum_{k=0}^{n-1} L^{n-1-k} r_k.$$

`ErrorBudget.trajectory_error_bound` proves the recursive enclosure for arbitrary normed additive groups. `constant_residual_envelope` proves its geometric finite-sum form for a constant defect cap. With nonexpansive propagation and a positive constant cap, a sufficient waiting window is:

$$E_n \le E_0 + n\eta, \qquad n \le \frac{\varepsilon - E_0}{\eta} \Longrightarrow E_n \le \varepsilon.$$

`nonexpansive_envelope` and `correction_window` establish the displayed arithmetic. Apply the trajectory theorem to a constant cap that bounds every actual defect before using this window. A correction resets E0 only if its post-correction error is independently bounded. A finite pivot budget says nothing about whether computation finishes inside this physical window.

PO-033 section 3.1 and PO-034 section 4.2 give the conditional continuous-time Duhamel estimate, including initial mismatch in PO-033:

$$|\Delta O(t)| \le \|O\|_\infty \left(E_0 + \int_0^t \|\mathcal{R}(s)\|_1 \, ds \right).$$

The residual here is the actual dynamical defect of the implemented interpolated trajectory. With a proved uniform residual cap $r > 0$ and observable norm $\|O\|_\infty > 0$, a sufficient waiting window is:

$$t \le \frac{\varepsilon/\|O\|_\infty - E_0}{r}.$$

If $r = 0$, do not use the divided expression: the supplied enclosure is constant in time and certifies tolerance whenever

$$\|O\|_\infty E_0 \le \varepsilon.$$

This conclusion is restricted to the interval on which the enclosure is valid. A zero residual cap does not repair an excessive initial-error bound. If the observable norm is zero, the displayed enclosure gives zero observable error; any nonnegative tolerance then suffices.

`duhamel_waiting_window` formalizes the scalar implication from a supplied enclosure and cap. The CPTP/unitary propagator theorem, continuous integral enclosure, and validated numerical quadrature are

not newly mechanized here. PO-033's sampled trapezoidal estimates explicitly remain uncertified; initial normal velocity alone cannot replace a uniform cap. PO-000 Theorem 5.10.A can sharpen observable bounds through certified edge residuals and Lieb–Robinson weights, provided its locality and trajectory hypotheses hold. Those spatial weights do not automatically apply to a reasoning state or arbitrary self-modification.

To connect a state enclosure to contextual data, declare the observation maps. `observation_radius` proves:

$$\sum_i |O_i(x) - O_i(y)| \le \left(\sum_i \|O_i\|\right) \|x - y\|.$$

`nearest_distance_lipschitz` proves the 1-Lipschitz comparison for attained distances. `uncertainty_adjusted_witness` then proves, for every feasible probability vector p and any feasible bounded witness:

$$\sum_i |e_i - \hat{e}_i| \le \rho \Longrightarrow \|Ap - e\|_1 \ge \max\{0, \langle y, \hat{e}\rangle + z - \rho\}.$$

The radius needs a justified observation/error or statistical coverage argument. The exact near-boundary fixture has margin 1/25000000; a radius at least that large erases this guaranteed positive margin. Exact arithmetic alone supplies no empirical confidence region. The review's sharper witness-norm factor, exact ball erosion, two-polytope certificates, and stochastic adaptive-transcript extensions remain separate prose/reference-code results in the supplied review, not newly claimed formal theorems in this edition.

Uncertain observations and the API boundary The public exact-table function `contextuality_lp.evaluate` requires normalized nonnegative rows and exact overlap compatibility. It returns `SIGNALLING` for the incompatible tables that finite sampling can produce. That is its intended input boundary, rather than evidence that noise can never occur.

The separate `evaluate_observed(scenario, uncertainty_radius)` route constructs deterministic columns from the declared outcome sets and contexts, retains the observed rows unchanged, and calls `solve_l1` on their flattened vector. It still requires exact normalized nonnegative row frequencies. The supplied radius is nonnegative and bounds the total raw sum of absolute observation errors over all context/outcome entries; it is not a context average. A measurement or statistical argument must justify its coverage.

The returned `ObservationBound` contains the observed vector, assignments, radius, exact LP certificate and uncertainty-adjusted guaranteed margin. `verify_observed` rebuilds the columns and replays that certificate against the original scenario and supplied radius. The raw optimum concerns the observed vector. The adjusted margin lower-bounds the true vector's distance from the noncontextual hull only if that vector lies inside the justified uncertainty ball. Zero adjusted margin certifies neither a joint distribution nor contextuality. The result has no classification field for the unknown true table.

For the review's exact example, start with a classical mixture and move probability 1/100 in one context from outcome (0,0) to (1,0). The rows remain distributions but their overlaps disagree. The exact-table API refuses with `SIGNALLING`. The observed-vector optimum is 1/50, and the known raw observation error is also 1/50, so the adjusted guaranteed margin is zero. The known classical truth in this example is additional fixture data; it is not inferred from the zero margin. Run `python code/uncertain_observation_example.py` to reproduce the full observed vector, certificate and bound.

This route neither fits nor renormalizes the data to force compatibility. If a caller replaces the observed vector by $\bar{e}$, a generally sufficient radius about that replacement is

$$\rho + \|\bar{e} - \hat{e}\|_1,$$

by the triangle inequality. A smaller replacement radius requires a separate justification. For arbitrary exact targets that are not probability rows, use the lower-level `solve_l1` with the declared deterministic columns and retain the same explicit uncertainty argument.

Finally, a short-time leakage lower bound becomes a correction-capacity obstruction only after bounding what correction can remove in the same metric:

$$D(t) \geq \nu t - Kt^2, \quad C(t) \leq ct, \quad R(t) \geq D(t) - C(t) \Longrightarrow R(t) \geq (\nu - c)t - Kt^2.$$

`correction_capacity_lower_bound` and `threshold_missed` prove this implication and its strict-threshold consequence. PO-000 supplies a local correlation lower bound with K equal to 24 times the squared full incident interaction norm; PO-034 supplies different finite-carrier coefficients under its own Hamiltonian, jump, and dilation hypotheses. These quantities and domains must not be interchanged. A lower bound gives a forbidden window when it exceeds tolerance; it does not certify safety at other times, monotonic error growth, or a universal deadline for correcting reasoning.

3.3 Changing representations without erasing accumulated error

The supplied *Minds Under Partial Observation* packet extends the same accounting question to learning and memory changes. A changed decoder must be compared in a common state space. For a continuous reference trajectory, an approximation that is absolutely continuous between finitely many representation switches, an integrable one-sided stability rate, an integrable actual differential-defect cap and verified switch jumps, the conditional scalar comparison argument bounds the actual trajectory error by $e(t) = \|x(t) - \hat{x}(t)\| \leq E(t)$, provided E0 bounds the initial error. The following equality defines the upper envelope E(t); it need not equal the actual error. Take the post-switch value at a switch instant.

$$\begin{aligned} E(t) &= e^{\Lambda(t,0)} E_0 + \int_0^t e^{\Lambda(t,s)} \eta(s)\, ds \\ &\quad + \sum_{\tau_j \leq t} e^{\Lambda(t,\tau_j)} \epsilon_j, \\ \Lambda(t,s) &= \int_s^t \lambda(u)\, du. \end{aligned}$$

The reference and approximation must exist on the entire declared interval and remain in a domain where those bounds hold; negative stability rates are allowed. A decoder depending on changing model parameters contributes its parameter-derivative term to the actual defect. Numerical, interpolation and reference-model mismatch also belong in that defect. An actual system reset needs its own Lipschitz/reset-error contract; a policy change needs comparison under the same realized actions or a closed-loop simulation result. A state-preserving representation enrichment can reduce future defect production while leaving accumulated uncertainty unchanged.

This hybrid continuous comparison is a mathematical argument imported from the supplied packet, not a new Lean ODE theorem. `ErrorBudget` retains its explicitly conditional continuous enclosure and discrete results. The current RSI companion proves exact commuting memory updates and finite-state gate invariants. The separate historical recovery edition's changing-representation and nonlinear examples are not imported into the current proof estate. A zero leading composition defect does not prove exact closure at higher order, and a nonzero discarded component need not return to a retained readout. These scalar budgets concern declared state and observation errors.

4. Internal checks and self-certification

4.1 Ledger validity and completeness

A linked ledger contains payloads and stored digests. A verifier checks that every digest agrees with the previous digest and the payload. Every prefix of a verified ledger still verifies. `ResultIII.prefix_verifies` proves this for an arbitrary digest function. Consequently an internal validity check alone cannot distinguish the whole ledger from a withheld suffix.

An externally expected length and tip can distinguish a particular full ledger from its proper prefixes when the data and integrity assumptions support that comparison. In the finite source example the full two-entry ledger passes against its expected tip and its one-entry prefix fails. This is a formal example, not a proof that any hash tip uniquely identifies any history.

Proposition 3 (formal counterexample). For a constant digest function, both the empty ledger and a nonempty valid ledger have the same tip and pass the same tip check. `Bridge.pinned_tip_can_collide` exhibits them. An external pin is useful evidence relative to a trusted commitment, expected extent, and suitable integrity assumptions. It is not an unconditional uniqueness theorem about arbitrary digest functions or a way to know whether the external source withheld records. Python uses SHA-256; Lean's illustrative digest is arithmetic. Neither implementation proves cryptographic collision resistance.

4.2 Relabeling

If an export is invariant under a relabeling, every verdict computed solely from that export is also invariant. `ResultIII.invariant_export_blind` proves the factorization statement. A verdict sensitive to the first token separates a concrete list from its relabeling, so dropping invariance makes the assertion false.

An internally detectable invariant defect remains detectable. Relabeling invariance does not imply that token computation cannot detect any quotient defect. Questions about what a token denotes require a supplied grounding map. This is another observation-fiber issue, with the missing information named rather than inferred from a general claim about semantics.

4.3 Logical self-reference

Tarski's undefinability and Löb's theorem concern specified formal languages and provability predicates. Their application to an agent requires hypotheses about its effective theory, arithmetic, coding, and derivability conditions. They are imported background, not Lean results in this package. A token stream, self-report, or level counter does not establish those hypotheses. The present finite examples do not prove a general incompleteness theorem for physical cognition, awareness, or arbitrary self-modification.

5. Provenance, permission, and safe delegation

The adopted strict policy uses proposer, evaluator, and authorizer identifiers. It requires distinct role identifiers, certificate rather than prose standing, and an admitted root distinct from proposer and evaluator. It refuses malformed identifiers and unadmitted roots. `ResultIV.pass_implies_policy` proves the corresponding conditions from a successful Lean-model evaluation; `delegated_standing_admitted` proves the converse existence of a receipt under its stated premises.

This is a policy theorem. The receipt's effect type contains only `none`; saying that it carries no authority is a fact about this type. Delegated standing is an input supplied under a regime, not a consequence of predictive competence. Distinct strings can name one controller; the deposited alias witness passes. The model does not authenticate identities, verify a cryptographic delegation chain, or establish institutional independence.

5.1 A gate can pass an unsafe action

The three-role function has no action-safety predicate. Attach the action 1 to a valid packet in a system where only action 0 is safe. The packet still passes. `Bridge.provenance_pass_can_be_unsafe` proves this concrete counterexample; the Python suite checks the same separation. No contradiction arises, because provenance and safety answer different questions.

The current RSI companion implements checked memory operations and a state-transition gate whose accepted next state satisfies a declared decidable invariant. Its proof covers every finite sequence from a safe initial state. Applying that gate to partially observed physical actions additionally requires a sound model of possible states and actions safe throughout the compatibility set. A provenance certificate cannot supply those semantic premises.

Conversely, contextuality alone is not an action veto. To obtain a decision impossibility one must specify the action set, the loss or safety relation, and which states remain compatible with the available report. If every compatible state has a common safe action, refusal is not forced by an inability to reconstruct the whole state. A physical action gate therefore needs this explicit compatible-state contract.

5.2 Memory under partial observation: reports and decisions

The mind-expansion packet is integrated here as an account of warranted claims and actions. A report is one observation channel of a specified agent; loss at that channel does not establish loss from the agent's retained state, tools or environment. Two complementary records can jointly answer a question neither answers separately; duplicating one record cannot restore its missing distinction. Pooling also requires communication, record identification and a usable decoder. This account separates record determination, effective computation, source justification and permission. Learning and intelligence concern performance over declared tasks and resources.

Theorem 3a (formal, report-only acceptance). For an arbitrary report map and correctness predicate, accepting a report whose whole fiber is correct is sound. Every sound report-only acceptance rule is contained in this rule. `MindContracts.whole_fiber_sound` and `whole_fiber_maximal` prove both claims for arbitrary types. This characterizes the greatest sound acceptance set, including vacuous empty fibers; it supplies no algorithm for deciding an arbitrary predicate on a fiber. The independent LP checkers are useful because their input relation is explicitly specified and computable. Distinct names for checkers or role holders alone do not establish operational independence.

Theorem 3b (formal, decision guarantee). If every action score differs from its true value by at most a common bound, choosing a maximal score gives

$$\begin{aligned} |V_a - s_a| \le b \ (\forall a), \quad s_a \le s_{\hat{a}} \ (\forall a) \\ \implies \quad V_a - V_{\hat{a}} \le 2b \ (\forall a). \end{aligned}$$

`score_regret` proves the assertion. `score_gap_unique` proves that a strict score gap greater than twice that bound against every other action makes the chosen action uniquely optimal. A state-error enclosure and a Lipschitz value model can justify a score bound by

$$b = KE + \zeta.$$

This last reduction needs a valid state enclosure, value-model Lipschitz bound and score-evaluation error cap. It does not supply an action permission or prove that the value model represents human interests. `upper_bound_does_not_force_failure` separately exhibits zero actual error under an upper bound above any nonnegative tolerance. Failure of an available certificate and actual task failure are different propositions.

An action-interface expansion may distinguish states previously merged by memory; the RSI companion's changing-action contracts already address that mathematical boundary. For LLM applications, declare whether the object is a base model, an interactive agent with tools, or a larger sociotechnical system. Then test retained distinctions, future task readouts and interventions at that boundary. Context-dependent answers alone do not establish Bell-style contextuality without identified measurements, contexts and compatible marginals.

Finite-horizon observability and response conditioning become operational tests after the input/output channels, units, noise floor and horizon have been declared. An uncoupled bank of memory registers can be fully observable, so observability alone does not establish causal coupling. Controlled studies can compare retained information with available reports, extend action interfaces, measure adapter costs and test intervention predictions. The supplied packet's further interpretations remain historical source proposals; these operational contracts need no additional experience premise.

The six packet files are preserved verbatim under `docs/source_inputs/mind_expansion`, with checked source hashes. Its six illustrative groups are independently replayed here through 19 standard-library assertions.

The optional two NumPy/SymPy checks and the packet's cited human studies are not independently verified by that replay. The floating finite-difference illustrations remain illustrations, distinct from exact LP certificates and generic Lean proofs. `docs/mind_expansion_integration.md` records the evidence boundaries.

6. Embedded knowledge, recovery and observation budgets

The world-internal rule treats agents, records, instruments and evaluators as interacting parts of one world. Record-relative determination quantifies over states compatible with a declared observation; it neither introduces a detached observer nor identifies an authorization status with truth. The shared constructive module `EmbeddedLimits.lean` proves factivity, positive introspection, own-record determination, exact task recovery on fibers, and the greatest stable relation preserving all future readouts. Both packages carry identical source bytes for that core and audit them independently. Paper 000 develops the memory interpretation and supplies executable checked update, recall and gate definitions.

6.1 Exact scalar recovery and memory capacity

Theorem 4 (formal, attained scalar minimax). Suppose each realized record fiber has declared lower and upper task extrema, their half-widths are at most a common radius, and a same-record pair attains twice that radius. Then a midpoint decoder attains the radius and every decoder's uniform error bound is at least that radius:

$$d(o) = \frac{\ell(o) + u(o)}{2}, \qquad R = \frac{1}{2} \max_{r(x)=r(y)} |f(x) - f(y)|.$$

`midpoint_error`, `fiber_pair_lower_bound` and `attained_fiber_minimax` prove the upper bound, lower bound and their combination over the reals. The Lean certificate theorem takes explicit bounds and an attaining pair; the exact finite generator computes these objects and its independent checker replays coverage, extrema, midpoint, radius and witness. This is not a theorem about arbitrary metric-valued tasks or nonattained infinite suprema. For one record containing 112 and 143, the certified optimum is $31/2$; retaining a separating record makes it zero.

Theorem 5 (formal, memory-state lower bound). Choose one representative of each of a finite family of pairwise future-distinguishable states. If an encoder, decoder and updater satisfy the two commuting equations of paper 000, the family injects into the memory state space. `future_classes_require_memory` therefore bounds the number of represented future classes by the number of memory states. A fixed-length binary representation must have enough codewords. Expected-length compression, uncertain observations and a changed future action family require distinct assumptions.

6.2 Statistical experiment simulation

For probability vectors, this section uses total variation as half of the raw L1 distance. The contextuality solver's raw mismatch elsewhere in the paper remains unhalved:

$$\mathrm{TV}(p, q) = \frac{1}{2} \sum_i |p_i - q_i|.$$

Theorem 6 (formal, bounded-loss transfer). Equal-mass finite vectors and a loss in the unit interval satisfy

$$0 \le L_i \le 1 \quad \Longrightarrow \quad \left| \sum_i p_i L_i - \sum_i q_i L_i \right| \le \mathrm{TV}(p, q).$$

`bounded_loss_tv` proves this by subtracting one half from the loss. The constant contribution cancels because the total masses agree, and each centered loss has absolute value at most one half. Probability nonnegativity is not needed for this algebraic bound; it is needed to interpret the vectors as distributions.

A row-stochastic kernel maps an observation of a source experiment to a simulated observation of a target experiment. `stochastic_simulation_nonnegative` and `stochastic_simulation_mass` prove the probability constraints under their explicit premises. `simulation_loss_transfer` proves that a uniform total-variation simulation error transfers every supplied unit-interval loss by at most that error, parameter by parameter. A fixed target randomized decision rule can be absorbed into its expected loss. The direction is source-to-target simulation:

$$\delta(E, F) = \inf_{K} \sup_{\theta} \mathrm{TV}(KE_\theta, F_\theta).$$

This definition motivates the next solver extension. The new Lean result is the forward risk-transfer inequality for a supplied kernel and certified error; an infimum-attaining deficiency optimizer and a converse comparison theorem are not deposited. The experiment-comparison literature distinguishes prior-weighted and uniform deficiency, and some papers use L1 units twice ours [10]. Task-specific exact memory is weaker than preservation of every decision problem. Quantum comparison needs the stated positivity, complete positivity and ancillary-system conditions [11]; classical bounded-loss transfer alone does not construct a quantum recovery channel.

6.3 Exact observation planning

Let a finite candidate set contain the actual world, and let each permitted query produce a deterministic binary answer without changing that world. A leaf checker certifies a readout error for every world remaining in its candidate set. Define feasibility at zero budget by the leaf checker; at the next budget, accept a checked leaf or a permitted query whose two branches are feasible at the preceding budget.

Theorem 7 (formal, query-budget equivalence). `possible_iff_bounded_plan` proves this executable Boolean recurrence true exactly when there exists a valid binary query tree with maximum depth at most the supplied natural budget. `rejected_budget_excludes_plan` gives the converse refusal meaning for that budget. The recursion terminates structurally on the budget. `informative_branch_card_lt` separately proves that a branch excluding a witnessed alternative has strictly smaller finite cardinality, justifying strict-refinement search termination.

`actual_world_retained` proves that filtering by the actual answer keeps the actual world. `valid_reaches_checked_leaf` proves that a valid plan reaches a checked leaf. `checked_leaf_error` combines these results with the supplied leaf-checker soundness premise to establish the declared error bound on the actual task answer. This is a correctness argument about nondestructive queries with an explicit leaf contract, not an assumption that observing is physically safe.

The exact Python planner accepts arbitrary finite answer labels and a rational scalar tolerance, returning an independently checked minimax tree. It specializes to the Lean binary model on the three-query fixture

$$q_0 = (0, 0, 1, 1), \quad q_1 = (0, 1, 0, 0), \quad q_2 = (0, 0, 0, 1), \quad f = (0, 1, 2, 3).$$

Querying the first coordinate partition, then the appropriate remaining query, identifies the task in two steps. Every fixed identifying panel needs all three queries. Five closed Lean fixture theorems check the two-step acceptance, one-step refusal, collision of every fixed pair, sufficiency of all three tests, and refusal when no queries are permitted. The checker also rejects a sound but nonoptimal tree, and tests retain an unresolved pair and a query family with no permitted distinguishing test. Finite algorithm checks do not prove universal Python-to-Lean refinement. Angluin's active-learning results likewise depend on a specified teacher/query resource [12].

Query depth is not elapsed time. The correction bounds in section 3.2 can be combined with a justified query-duration model: a plan must complete before the required enclosure margin is exhausted to be certified by that bound. An exhausted upper bound does not establish actual failure, and early correction may remain possible. This edition does not infer latency from tree depth or prove a physical deadline without a trajectory, observation and duration model.

6.4 Learning without a universal intelligence ceiling

The literature study treats capability as indexed by environment, task, observation, action, memory, computation and error. Fano bounds require distributional information premises; continual-learning bounds depend on pass and class restrictions; multiple passes and retained distinctions can constructively remove some obstructions. Logical induction supplies qualified introspection and self-trust results. The full primary-source catalog and scope notes are in `docs/embedded_literature.json`.

Adaptive validation is a further, separate premise. The inspected version 2 of Statistical Gödel Machines uses conditionally valid tests and predictable summable error allocations over every opened confirmation, including failures [13]. A non-improvement guarantee for its objective does not certify action safety. Confidence sequences and holdout-reuse results are imported, not formalized here. The current packages establish finite memory/gate and analytical certificate results, plus a bounded live-agent memory experiment; they do not establish an empirical intelligence benchmark.

7. Executable scope and reproduction

The standalone Python modules use the standard library and exact fractions. The inherited and bridge tests cover contextual supports, marginals, CHSH, observation maps and words, ledgers, relabeling, and the three-role policy. The LP tests check exact objectives, randomized one-dimensional hulls against their closed form, certificate component mutations, degenerate constraints, negative right-hand sides, signalling input, malformed data, and resource refusals. The Lean generator independently checks five exact solver outputs; finite fixture checks supplement the general duality proof.

The Lean package pins Lean 4.35.0 and a source-pinned offline Mathlib bundle. The four original result modules remain byte-for-byte identical to the precursor. `Bridge` supplies the RSI connections, `ContextualDuality` proves the general LP results, and `RationalCertificates` checks the generated rational data. `Simplex` implements the total budgeted solver and proves its pivot, return and accounting results; `CertifiedOutcomes` supplies the extended checked-outcome program and real-vector certificate soundness; `SimplexDriver` exposes its exact execution. `ErrorBudget` proves the conditional propagation, observation and correction-window statements. `MindContracts` supplies the report-fiber, decision and upper-bound results. `EmbeddedLimits` supplies the shared constructive knowledge/memory core; `EmbeddedQuantitative` proves scalar minimax, memory capacity and loss transfer; `QueryBudget` proves the executable binary feasibility recurrence, bounded-plan equivalence and leaf-error correctness. Each package-local theorem enters the named census; the full environment census additionally checks generated and private package declarations. Only `propext`, `Quot.sound`, and `Classical.choice` are admitted. Source hashes bind the audited modules and build configuration; changes after audit are refused. Vendored dependencies have their own source, license, module and artifact inventory.

The Lean, Python, and external Rust gate implementations remain separate models. No compiler or refinement proof ties their semantics together. The new Python boundary check refuses unknown basis strings; Lean's basis type already admits only its two constructors. Tests bind that documented boundary behavior, without claiming general cross-language equivalence.

Reproduction commands, from this package root, are:

```
python -m pytest code/tests -q
python tools/generate_lp_certificates.py --check
python tools/replay_simplex_implementation.py --replay
python tools/replay_review_corrections.py
python tools/replay_mind_expansion.py
python code/uncertain_observation_example.py
python verify_deposit.py --lean
python tools/replay_lean_axiom_census.py --replay
python tools/replay_full_axiom_census.py --check
python scripts/paper_spine.py verify
python tools/rebuild_release.py --check
```

The mathematical text, generated TeX, and UTF-8 text share an explicit equation registry. The PDF is freshly compiled from that source; byte bindings and rendering checks are conversion checks, not proofs of

mathematical truth. The release ZIP and both manifests are checked against the package tree. No source from the sibling paper is needed to run this package.

Version 1.8.3 repairs three API and reproduction boundaries identified by the October 23 review. The optimal-certificate checker normalizes original integer or Fraction problem inputs and defensively validates its witness, as the infeasibility and unboundedness checkers already do. Malformed standard and defect certificates return false. Nonnegative integer pivot counts are schema metadata; their values are not evidence of optimality. The feasibility inequalities and zero-gap criterion are unchanged.

Full-census check mode compiles the existing sealed source without rewriting it, preserving either LF or CRLF bytes. Only explicit write mode regenerates the census. The complete verifier checks the original manifest expectations again after proof replay. Release reproduction uses the pinned publication date, while an intentional edition change requires the explicit new-release mode. A staged envelope must pass validation before replacing the active ZIP and manifests; ordinary replacement exceptions restore the previous envelope. These are executable integrity checks, not a proof of crash-atomic multi-file publication. The response and verification scope are recorded in `docs/review_response_2026_10_23.md`. No theorem statement changes in this patch.

8. Limits and research directions

General finite contextuality LP duality and an exact solver are now included. The source's additional support classifications and nerve computations remain historical external results unless independently represented in the deposited checks. The budgeted Lean simplex is total and success-sound; unbudgeted Bland completeness and universal Python refinement remain unformalized. General probabilistic privacy, stochastic interventions, real-system identity binding, cryptographic commitments, and verification of a self-modifying model remain application obligations. The Tarski and Löb results remain imported.

Checked infeasibility and unboundedness are now implemented, with soundness proved for accepted certificates. Proving unbudgeted search completeness, removing final replay through complete phase invariants, or proving universal Python refinement remains future work. The companion now proves its finite implemented memory and gate definitions. Refinement of a deployed runtime and adequacy of its safety predicate remain open.

The constructive target is safe delegation under declared observations and updates. The next deployment result should connect persisted memory, fresh checked recall and authenticated action dispatch to the companion's implemented transition model. That refinement must establish the source bindings, future-task contract and physical action-safety relation. Descriptive self-reports and successful provenance checks alone supply neither proof.

9. Persistent memory in the live agent

The persistent experiment binds a declared task, model, action set, exact key/value encoding and update rule. Its categorical residual is zero exactly when all recorded observations sharing a key agree. Every recall and proposed update rechecks that residual, source hashes, exact source membership and the contract. Accepted facts enter the repository agent's user context before prompt binding. Rejected updates retain the previous archive; failed recalls prevent dispatch. SQLite transactions and a persisted database carry observations between processes.

The captured exact-use trial used four separate processes: an uninformed baseline returned UNKNOWN, the agent extracted a supplied synthetic observation, and two later processes used the stored value with different requested suffixes. An earlier arithmetic trial failed after correct recall. Thus persistent access and correct downstream reasoning remain different properties. Both outcomes and raw replies are retained, with independent offline replay of prompt bindings and source pins. This is bounded evidence from one model, not a general learning or intelligence benchmark. The connection is to the repository agent facade; it does not modify Codex's global context or model parameters.

The eight Lean statements in `LiveMemory` prove source-supported, consistent updates and recalls, preservation

of old entries, refusal behavior and recall after an explicitly exact reload. SQLite crash recovery and the live model's use of context are tested operationally, not proved by these abstract statements. Source adequacy remains a premise. Implementation, commands and full limitations are in `docs/live_memory.md`; captured-trial replay uses `tools/replay_live_memory_evidence.py`.

The retained v1.8.2 verification records that the repaired quantum library passed all 712 tests in release mode, with no failures, ignored cases or filtered cases. This includes the three prior numerical failures, all ten previously unfinished MUB tests, and three new admission-boundary regressions. Exact PSD of the supplied binary Choi matrix replaces a rounded eigenvalue-sign decision. Floating-point Hamiltonian evolution uses a separate tolerance-checked numerical propagator; exact admission remains strict, and numerical evolution retains norm drift. The rebuilt native budget host passes all 24 gateway-and-memory tests, including both previously blocked cases; five quantum integration binaries pass 57 tests. These are scoped checks of the recorded v1.8.2 sources, not new v1.8.3 executions. The earlier multi-crate audit remains historical, and these synthetic gateway tests add no new live model trial or universal implementation-refinement proof. The current source-bound build and test records are in `docs/numerical_verification.json`; `docs/native_gateway_verification.json` preserves the preceding partial run.

The companion Projection Obstruction extensions separate a measured residual from an error enclosure. PO-000 section 5.18 transfers uncertainty into onset and correction-window bounds; PO-033 section 3.5 accounts for integration defects and finite correction jumps; PO-034 section 4.3 gives density-state product-defect uncertainty at most (k+1) times the trace-norm state error. Their exact scalar witnesses check supplied-bound arithmetic. The continuous statements are written proofs with declared premises, not new Lean results or automatic error certificates for recalled memories.

References

The precursor's full bibliography and verification markers are preserved in `docs/precursor_references.md`. References 1–6 are carried from the supplied sources. The primary publisher record for Bland [7] and the solver-certificate manual [8] were checked for this correction; no general historical novelty claim follows.

1. Carroll, J. H. (2026). *Relational Reasoning, Memory, and Embedded Knowledge*. Version 1.2.3. Current RSI paper 000. The distinct historical recovery edition 1.4.0 remains identified in `docs/source_lineage.json`.
2. Carroll, J. H. (2026). *Conditional Limits on Global Prediction, Self-Knowledge and Self-Certification for Physical Reasoners*. Version 1.2.0. Precursor manuscript retained as `docs/precursor_monograph.md`.
3. Abramsky, S. and Brandenburger, A. (2011). *The sheaf-theoretic structure of non-locality and contextuality*. New Journal of Physics 13, 113036. doi:10.1088/1367-2630/13/11/113036.
4. Popescu, S. and Rohrlich, D. (1994). *Quantum nonlocality as an axiom*. Foundations of Physics 24, 379–385. doi:10.1007/BF02058098.
5. Tarski, A. (1936). *Der Wahrheitsbegriff in den formalisierten Sprachen*. Imported background at the scope stated in section 4.3.
6. Löb, M. H. (1955). *Solution of a problem of Leon Henkin*. Journal of Symbolic Logic 20, 115–118. Imported background at the scope stated in section 4.3.
7. Bland, R. G. (1977). *New Finite Pivoting Rules for the Simplex Method*. Mathematics of Operations Research 2(2), 103–107. Primary publisher record. Using the rule does not mean its unbudgeted finiteness proof has been mechanized here.
8. Fischer, K., Gärtner, B., Schönherr, S. and Wessendorp, F. *CGAL Linear and Quadratic Programming Solver: User Manual*, Solution Certificates. Primary documentation. Primary background for checked infeasibility and unboundedness; the equality-form witnesses used here have their own Lean soundness proofs.
9. *Minds Under Partial Observation* (2026-10-21), user-supplied research expansion, preserved in `docs/source_inputs/mind_expansion`. Source proposals and constructed examples; no independent empirical or novelty claim.
10. van Rooyen and Williamson (2014). Le Cam meets LeCun: deficiency and generic feature learning. Prior-weighted and uniform forms and norm conventions must be distinguished.
11. Buscemi (2012). Comparison of quantum statistical models.

12. Angluin (1987). Learning regular sets from queries and counterexamples.
13. Wu et al. (2026 revision). Statistical Gödel machines. Inspected version 2, September 17, 2026.